# Teamup Integration

## 1. Overview

### 1.1 Description

**Teamup Integration** connects **Teamup** (shared calendar platform) with **Newo Platform**.

It enables:

- Checking real-time availability slots across configurable date windows
- Booking appointments with automatic sub-calendar assignment
- Cancelling existing appointments
- Auto-detection of sub-calendars during setup
- Auto-provisioning of booking/cancellation scenarios to the workflow canvas

This integration is designed for **businesses and teams** who need **automated appointment scheduling through a conversational AI agent** (voice or chat) using Teamup as their calendar system.

### 1.2 Use Cases

- When a client asks about available slots → Check Teamup calendar for free time slots within the configured availability window
- When a client wants to book an appointment → Create an event in the Teamup calendar with title, notes, and duration
- When a client wants to cancel an appointment → Delete the event from the Teamup calendar by event ID
- When a conversation starts → Optionally preload nearest available slots into the agent's prompt context
- When the project is published → Auto-detect sub-calendars and configure booking/cancellation scenarios

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Check Availability | ✅ | Configurable window (default: 7 days) and slot duration |
| Book Appointment | ✅ | With title, notes, duration; auto-assigns sub-calendar |
| Cancel Appointment | ✅ | By event ID via DELETE request |
| Silent Availability Preload | ✅ | Optional auto-check on conversation start |
| Sub-Calendar Auto-Detection | ✅ | Fetches active sub-calendars during setup |
| Canvas Scenario Provisioning | ✅ | Auto-adds Speed-to-Lead, Cancellation, and Scheduling scenarios |
| Configurable Timezone | ✅ | IANA timezone for all date/time operations |
| Configurable Slot Duration | ✅ | Default 30 minutes, configurable per project |
| Rescheduling | ❌ | Cancel + rebook workflow (no direct reschedule API) |

| | | |
|---|---|---|
| Multi-Location Support | ❌ | One sub-calendar per instance; use multiple instances |
| Webhook Support | ❌ | No inbound webhook handling |

## 2. Requirements

Before installation:

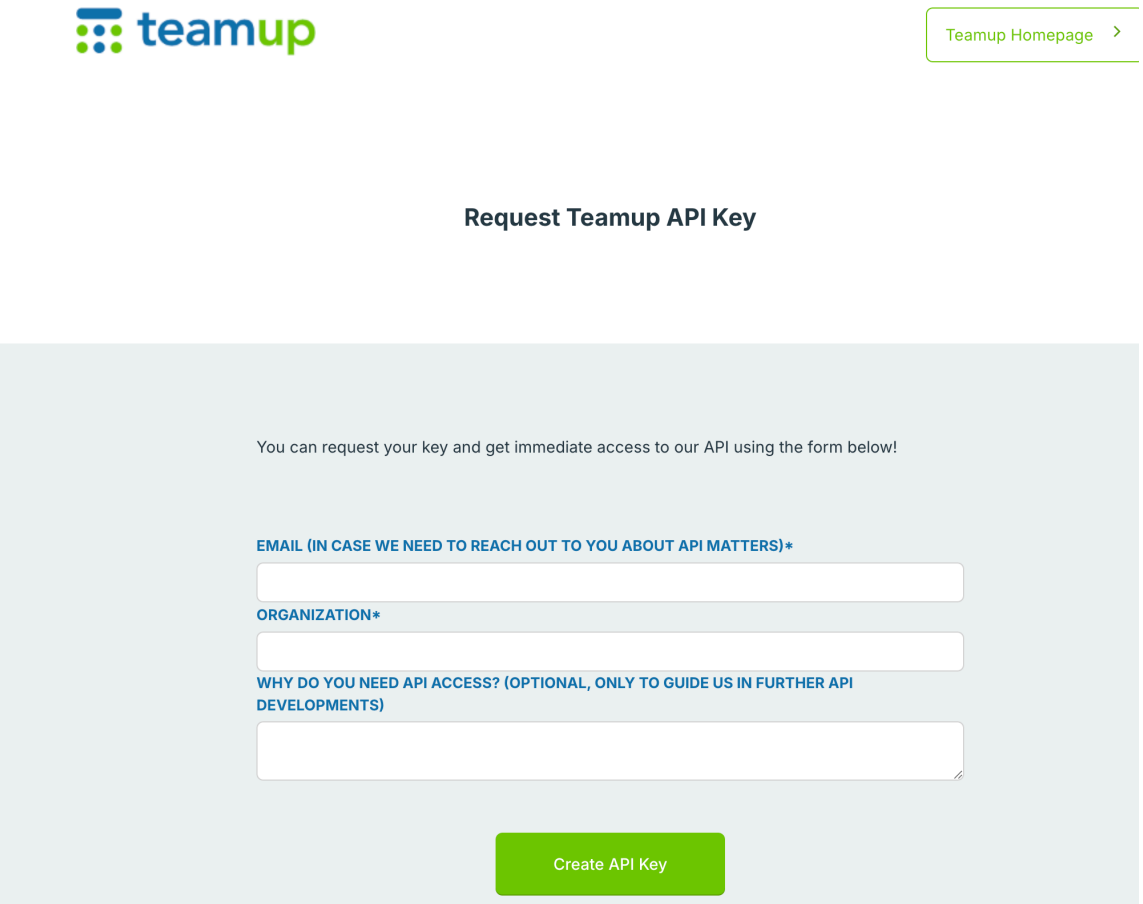
- Active **Teamup** account with API access
- **Teamup API Key** (request at https://teamup.com/api-keys/request)
- **Teamup Calendar Key** (the `ks...` key from your calendar URL that determines permission level)

## 3. How to Setup

### 3.1 Step 1 — Get Teamup API Key

1. Go to **https://teamup.com/api-keys/request**
2. Fill in the form: your email, organization name, and optionally describe the use case
3. Click **Create API Key**
4. Copy the API key and store it securely

---

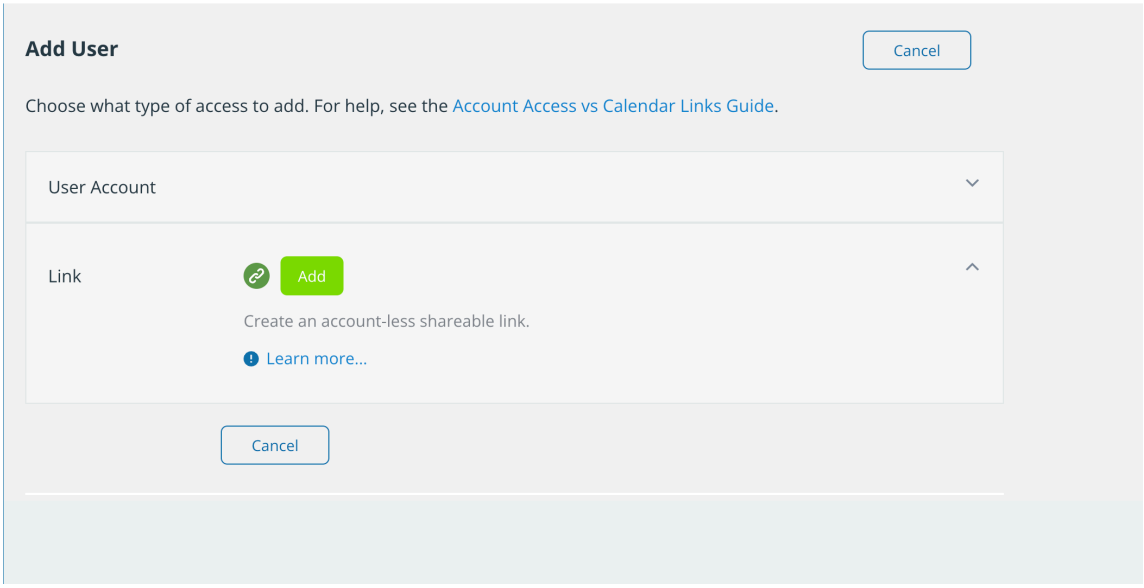



### 3.2 Step 2 — Create a Calendar Link with Modify Permissions

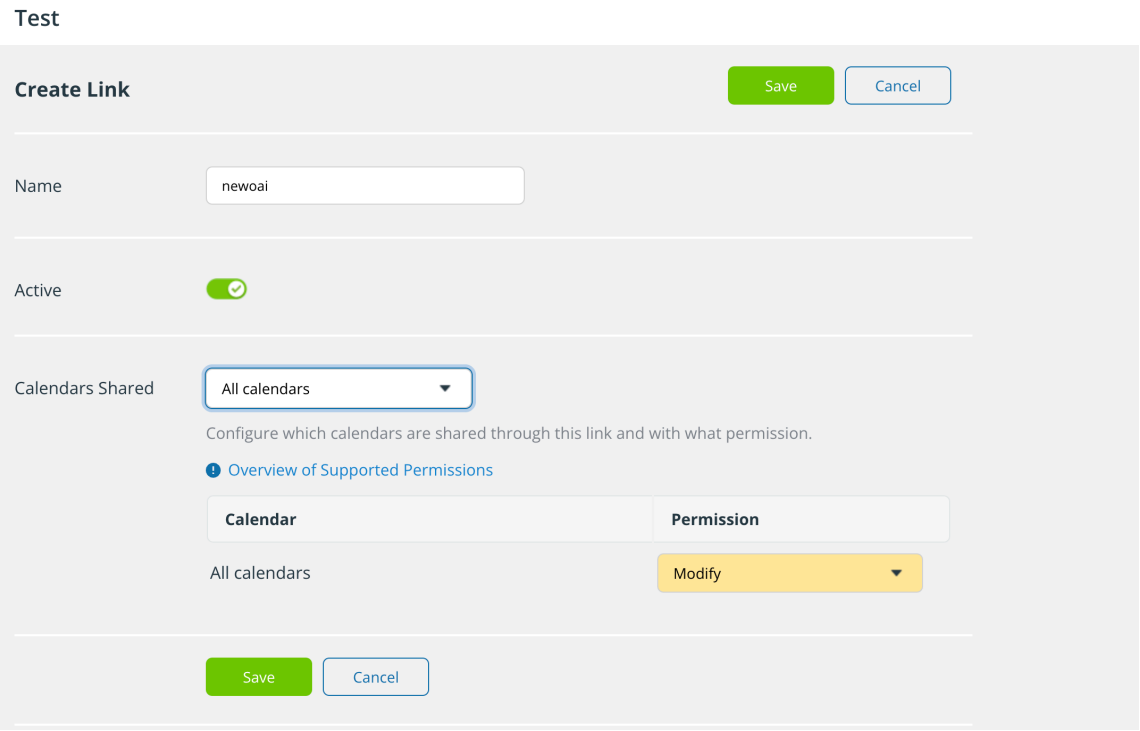
The Calendar Key ( `ks...` format) controls API access scope and permissions. You need a key with **Modify** permission for booking and cancellation to work.

1. Open your Teamup calendar
2. Go to **Settings** (gear icon) → **Sharing**
3. Click **Add User** → expand the **Link** section → click **Add**

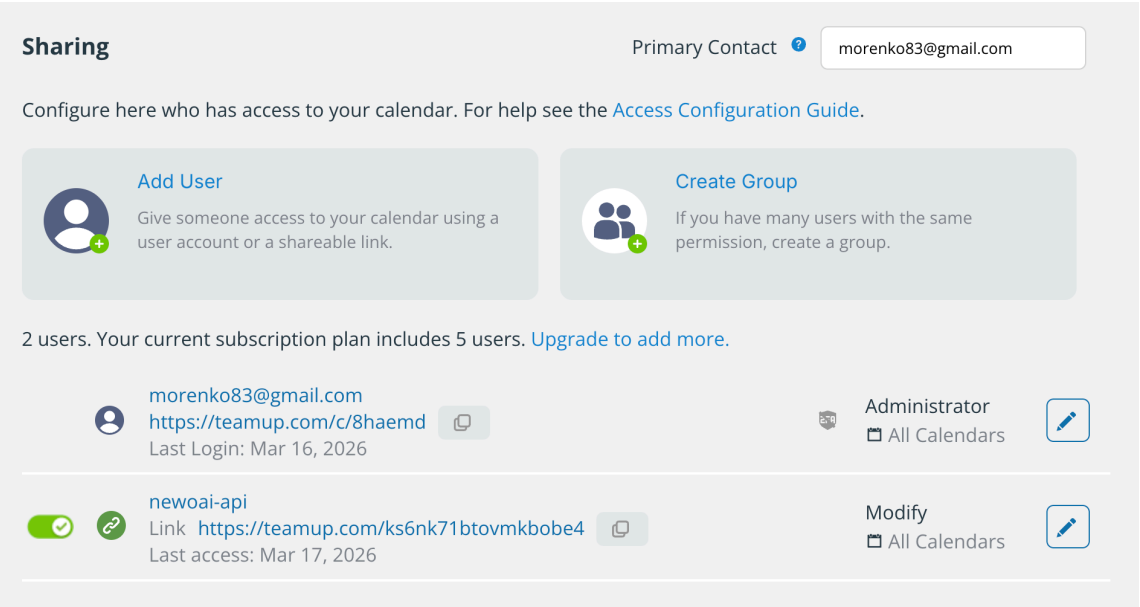
Test




4. In the **Create Link** dialog:
   - Set a **Name** (e.g., `newoai` )
   - Ensure **Active** is toggled on
   - Under **Calendars Shared**, select **All calendars** (or specific ones)
   - Set **Permission** to **Modify** (required for booking/cancellation)
   - Click **Save**




5. After saving, the calendar key appears in the Sharing list as a URL:
   `https://teamup.com/ks6nk71btovmkbobe4`
   - The part after `/` is your **Calendar Key**: `ks6nk71btovmkbobe4`



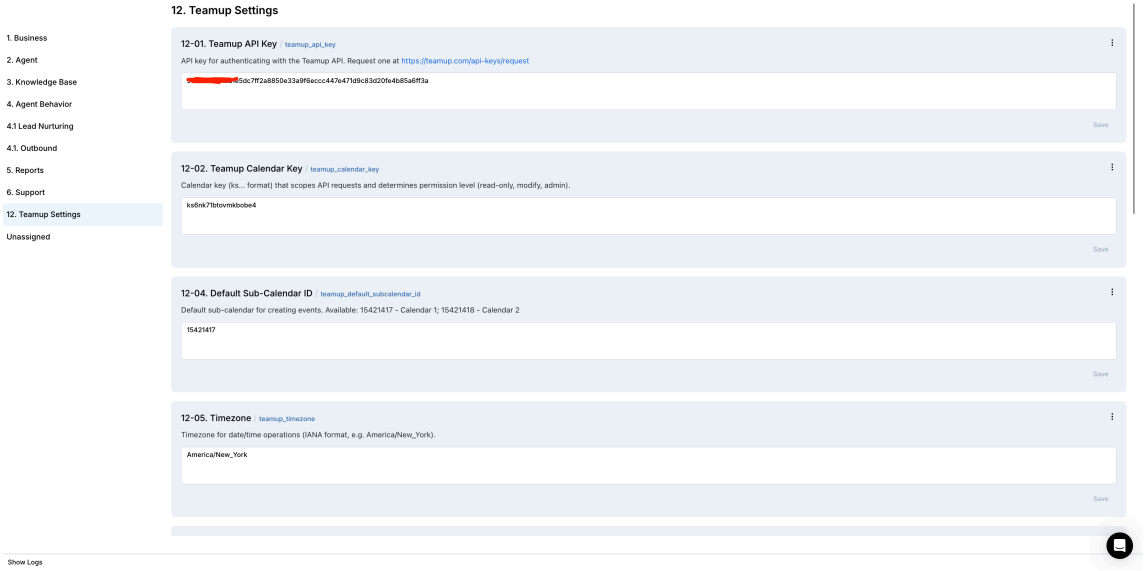

> ***Permission levels:***
>
> - ***Read-only*** *— only availability checking will work*
> - ***Modify*** *— availability, booking, and cancellation all work*

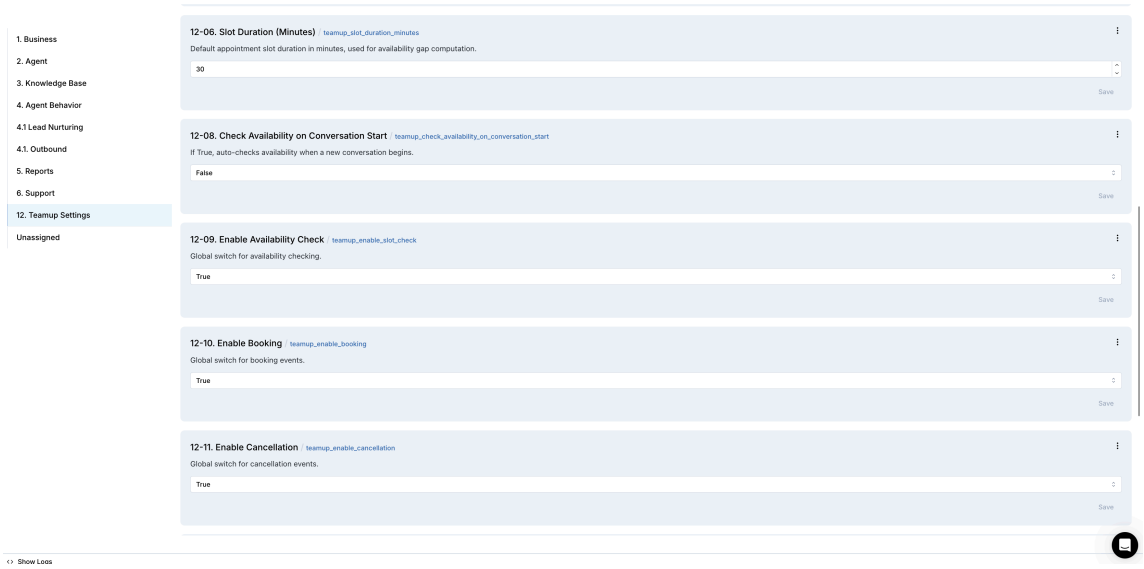
> - ***Admin*** *— full access including calendar settings*

### 3.3 Step 3 — Connect in Newo Platform

1. Open **Newo → Projects**
2. Navigate to **Builder / Attributes** → section **12. Teamup Settings**
3. Set the required attributes:

| Attribute | Required | Description |
|---|---|---|
| `teamup_api_key` | ✅ | API key from Step 1 |
| `teamup_calendar_key` | ✅ | Calendar key (`ks...`) from Step 2 |
| `teamup_base_url` | ❌ | API base URL (default: `https://api.teamup.com`) |
| `teamup_default_subcalendar_id` | ❌ | Default sub-calendar ID for events. Auto-detected if only one exists |
| `teamup_timezone` | ❌ | IANA timezone (default: `America/New_York`) |
| `teamup_slot_duration_minutes` | ❌ | Slot duration in minutes (default: `30`) |
| `teamup_show_for_days` | ❌ | Availability window in days (default: `7`) |
| `teamup_check_availability_on_conversation_start` | ❌ | Auto-check availability on new conversation (default: `False`) |
| `teamup_enable_slot_check` | ❌ | Enable availability checking (default: `True`) |
| `teamup_enable_booking` | ❌ | Enable booking capability (default: `True`) |
| `teamup_enable_cancellation` | ❌ | Enable cancellation capability (default: `True`) |
| `teamup_setup_scenarios` | ❌ | Auto-add default scenarios to canvas (default: `True`) |
| `teamup_override_agent_attributes` | ❌ | Override ConvoAgent booking schemas and feature flags (default: `True`) |




4. Click **Save + Publish All**
5. On publish, the SetupFlow automatically:
    - Creates the `teamup_connector` HTTP connector
    - Sets ConvoAgent booking/availability/cancellation feature flags
    - Injects payload schemas for availability, booking, and cancellation tools
    - Fetches active sub-calendars from Teamup API
    - Auto-selects default sub-calendar if only one exists
    - Provisions Speed-to-Lead and Cancellation scenarios to the workflow canvas

# 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

## 4.1 How the Integration Works

The integration operates based on **Triggers and Actions** via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Teamup

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| Project Publish | `project_publish_finish` | Initializes integration, creates connectors |
| Canvas Built | `gm_workflow_builder_canvas_built` | Provisions scenarios and intent types to canvas |
| Check Availability | `get_available_slots` | Queries available time slots from Teamup |
| Session Started | `session_started` | Optional auto-check availability on conversation |
| Book Slot | `book_slot` | Creates an event in Teamup calendar |
| Cancel Booking | `convo_agent_cancel_booking` | Deletes an event from Teamup calendar |
| Execute Request | `teamup_execute_request` | Internal: dispatches HTTP requests via connector |
| HTTP Success | `result_success` (http/teamup_connector) | Internal: handles HTTP response routing |
| HTTP Error | `result_error` (http/teamup_connector) | Internal: handles HTTP error routing |

**Available Actions**

- **GET /subcalendars** — Fetch active sub-calendars during setup
- **GET /events** — Query events within a date range for availability computation
- **POST /events** — Create a new calendar event (booking)
- **DELETE /events/{id}** — Delete a calendar event (cancellation)

## 4.2 Architecture Diagrams

**Overall Sequence**

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as TeamupIntegration
    participant API as Teamup API

    Note over I: Setup (on publish)
    A->>I: project_publish_finish
    I->>I: Create teamup_connector
    I->>I: Set feature flags & schemas
    I->>API: GET /{calendarKey}/subcalendars
    API-->>I: subcalendars list
```

```
    I->>I: Auto-select default subcalendar

    Note over I: Canvas Setup
    A->>I: gm_workflow_builder_canvas_built
    I->>I: Add library items (scenarios + intents)
    I->>A: gm_canvas_add_item_batch

    Note over U,API: Check Availability
    U->>A: "What slots are available?"
    A->>I: get_available_slots {date, time}
    I->>API: GET /{calendarKey}/events?startDate=...&endDate=...
    API-->>I: existing events
    I->>I: Compute free slots (gap analysis)
    I->>A: result_success {availability}
    A->>U: "Here are the available slots..."

    Note over U,API: Book Appointment
    U->>A: "Book me at 2pm tomorrow"
    A->>I: book_slot {bookingDateTime, title}
    I->>API: POST /{calendarKey}/events {body}
    API-->>I: created event {id}
    I->>A: result_success {booking_id}
    A->>U: "Your appointment is booked!"

    Note over U,API: Cancel Appointment
    U->>A: "Cancel my booking"
    A->>I: convo_agent_cancel_booking {booking_id}
    I->>API: DELETE /{calendarKey}/events/{id}
    API-->>I: undo_id
    I->>A: result_success
    A->>U: "Your booking has been cancelled."
```

**SetupFlow**

```
flowchart TD
    E["project_publish_finish"] --> CHECK{"registry_item_idn ==
teamup_integration?"}
    CHECK -->|"No"| SKIP["Return()"]
    CHECK -->|"Yes"| ATTRS["Set project attributes\n(API key, calendar key,
timezone, etc.)"]
    ATTRS --> CONN["Create teamup_connector\n(HTTP connector)"]
    CONN --> INIT["_init()\nSet feature flags, create setup persona"]
    INIT --> SCHEMA["_injectSchemaAttributes()\nSet
availability/booking/cancellation schemas"]
    SCHEMA --> CREDS{"API key & calendar key present?"}
    CREDS -->|"No"| DONE["Setup complete (partial)"]
    CREDS -->|"Yes"| SUBCAL["GET /{calendarKey}/subcalendars"]
    SUBCAL --> SUCCESS{"Subcalendars found?"}
    SUCCESS -->|"Yes"| AUTOSEL["Auto-select if single subcalendar\nPopulate
possible_values metadata"]
    SUCCESS -->|"No"| LOG["Log: no subcalendars found"]
```

```
    AUTOSEL --> DONE2["Setup complete"]
    LOG --> DONE2
```

**CanvasSetupFlow**

```
flowchart TD
    E["gm_workflow_builder_canvas_built"] --> CHECK{"teamup_setup_scenarios == True?"}
    CHECK -->|"No"| SKIP["Return()"]
    CHECK -->|"Yes"| LIB["SetupLibrarySkill()\nAdd scenarios & intents to library"]
    LIB --> DEL["Delete default industry scenarios\n(beauty, home service, etc.)"]
    DEL --> ITEMS["Collect canvas items:\n- Speed-to-Lead scenario\n- Cancellation scenario\n- Scheduling scenario\n- Cancellation intent\n- Existing Client intent\n- Reschedule intent\n- Appointment intent"]
    ITEMS --> ADD["gm_canvas_add_item_batch"]
```

**UtilsFlow**

```
flowchart TD
    E["teamup_execute_request"] --> PARSE["Parse payload from event"]
    PARSE --> EXEC["_executeRequest()\nBuild headers with Teamup-Token"]
    EXEC --> HTTP["SendCommand: send_request\nvia teamup_connector"]
    HTTP --> RESP{"HTTP Response"}
    RESP -->|"result_success"| RCHECK{"Status 401/403\nor error in body?"}
    RCHECK -->|"Yes"| ERR["teamup_result_error"]
    RCHECK -->|"No"| OK["teamup_result_success"]
    RESP -->|"result_error"| ERR
    RESP -->|"runtime_error"| SYSERR["Log error via DUMMY"]
```

**AvailabilityFlow**

```
flowchart TD
    E["get_available_slots"] --> GATE{"teamup_enable_slot_check == True?"}
    GATE -->|"No"| SKIP["Return()"]
    GATE -->|"Yes"| PARSE["Parse date, time, duration from payload"]
    PARSE --> VALIDATE{"base_url, calendar_key,\ndate, time present?"}
    VALIDATE -->|"No"| ERR["_checkAvailabilityError()\nresult_error"]
    VALIDATE -->|"Yes"| CALC["Calculate date range\n(start_dt + show_for_days)"]
    CALC --> API["GET /{calendarKey}/events\n?startDate=...&endDate=...\n&subcalendarId[]=..."]
    API --> BOOKED["Parse booked time ranges"]
    BOOKED --> GAPS["_getAvailableTimeSkill()\nCompute free slots via gap analysis"]
    GAPS --> OUT["result_success\n{availability: [{location, availableTimeSlots}]}"]
```

```
flowchart TD
    E2["session_started"] --> AUTO{"teamup_check_availability_on_conversation_start == True?"}
    AUTO -->|"No"| SKIP2["Return()"]
```

```
    AUTO -->|"Yes"| NOW["Set date/time to current moment\nin configured timezone"]
    NOW --> CHECK["_checkAvailability(parameters)"]
```

**BookingFlow**

```
flowchart TD
    E["book_slot"] --> GATE{"teamup_enable_booking == True?"}
    GATE -->|"No"| SKIP["Return()"]
    GATE -->|"Yes"| PARSE["Parse bookingDateTime, title,\nnotes, duration from
payload"]
    PARSE --> VALIDATE{"base_url, calendar_key,\nbookingDateTime, title present?"}
    VALIDATE -->|"No"| ERR["_createAppointmentError()\nresult_success {success:
false}"]
    VALIDATE -->|"Yes"| SUBCAL{"subcalendar_id configured?"}
    SUBCAL -->|"No"| ERR
    SUBCAL -->|"Yes"| BUILD["Build event body:\nsubcalendar_ids, title, start_dt,
end_dt,\ntz, notes, who (phone)"]
    BUILD --> API["POST /{calendarKey}/events"]
    API --> RESULT{"Event created?"}
    RESULT -->|"Yes"| OK["result_success\n{booking_id: event.id}"]
    RESULT -->|"No"| FAIL["result_success\n{success: false}"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"teamup_enable_cancellation == True?"}
    GATE -->|"No"| SKIP["Return()"]
    GATE -->|"Yes"| PARSE["Parse booking_id from\npayload.meta.booking_id"]
    PARSE --> VALIDATE{"booking_id present?"}
    VALIDATE -->|"No"| ERR["_cancelAppointmentError()\nresult_success {success:
false}"]
    VALIDATE -->|"Yes"| API["DELETE /{calendarKey}/events/{booking_id}"]
    API --> OK["_cancelAppointmentSuccess()\nresult_success {success: true}"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish the project and verify sub-calendars are detected (check logs for `[SETUP] Found N sub-calendars`)
2. **Check Availability**: Ask "What times are available this week?" — verify the agent returns grouped time slots by date
3. **Book Appointment**: Ask "Book me at 2pm tomorrow" — verify the event appears in your Teamup calendar

4. **Cancel Appointment**: Ask "Cancel my booking" — verify the event is removed from your Teamup calendar

**If no action occurs:**

- Ensure `teamup_api_key` and `teamup_calendar_key` are set correctly
- Verify the calendar key has **modify** or **admin** permissions (not read-only)
- Confirm `teamup_default_subcalendar_id` is set (auto-detected if single sub-calendar)
- Ensure `teamup_enable_slot_check`, `teamup_enable_booking`, `teamup_enable_cancellation` are `True`
- Confirm the project was re-published after configuration changes
- Check that `teamup_override_agent_attributes` is `True` for ConvoAgent tool gating

> *Note: This integration performs real operations in Teamup. Changes made through the agent are reflected in the live calendar.*

## 5. FAQ

**Q: How does availability checking work?** A: The integration fetches all existing events from the Teamup calendar within the configured date range, then computes free slots by checking for gaps between booked events. Slots are grouped by date and returned in 12-hour format (e.g., "02:00 PM").

**Q: What happens if no sub-calendar ID is configured?** A: During setup, the integration auto-detects sub-calendars via the API. If exactly one active sub-calendar exists, it is auto-selected. If multiple exist, the user must manually set `teamup_default_subcalendar_id`. Booking will fail without a sub-calendar ID.

**Q: Can I use a read-only calendar key?** A: Only for availability checking. Booking and cancellation require a calendar key with **modify** or **admin** permissions. The integration checks for 401/403 errors and routes them as auth failures.

**Q: Does the integration support rescheduling?** A: Not directly. Rescheduling is handled via a cancel + rebook workflow: the CanvasSetupFlow provisions a "Reschedule" intent type that routes through the cancellation scenario first, then proceeds to the scheduling scenario.

**Q: What timezone is used for slot computation?** A: The timezone configured in `teamup_timezone` (default: `America/New_York`). All date/time parsing and slot computation uses this timezone via Python's `zoneinfo` module.

**Q: What scenarios are auto-added to the canvas?** A: Three scenarios (Speed-to-Lead Outbound Call, Canceling Appointment via Agent, Scheduling Appointment via Agent) and three intent types (Cancellation via Agent, Existing Client Appointment, Reschedule or Modification). Default industry-specific scenarios (beauty, home service) are removed.

**Q: How is the booking stored for cancellation lookup?** A: When a booking is created, ConvoAgent stores the `booking_id` (Teamup event ID) in the persona's `bookings` attribute. During cancellation, ConvoAgent retrieves the booking and passes `meta.booking_id` to the CancellationFlow.